МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту



Лабораторна робота № 2

з дисципліни «Обробка зображень методами штучного інтелекту»

Виконав:

Студент групи КН-415

Лозовий Олег

Викладач:

Пелешко Д.Д.

Львів – 2025 р.

Тема: Суміщення зображень на основі використання дескрипторів

Мета роботи: навчитись вирішувати задачу суміщення зображень засобом видобування особливих точок і використання їх в процедурах матчінгу.

Варіант 12 (1)

Основне завдання (обов'язкове)

Вибрати з інтернету набори зображень з різною контрастністю і різним флуктуаціями освітленості. Для кожного зображення побудувати варіант спотвореного (видозміненого зображення). Для кожної отриманої пари побудувати дескриптор і проаналізувати можливість суміщення цих зображень і з визначення параметрів геометричних перетворень (кут повороту, зміщень в напрямку x і напрямку y).

1. SIFT

joe.jpg



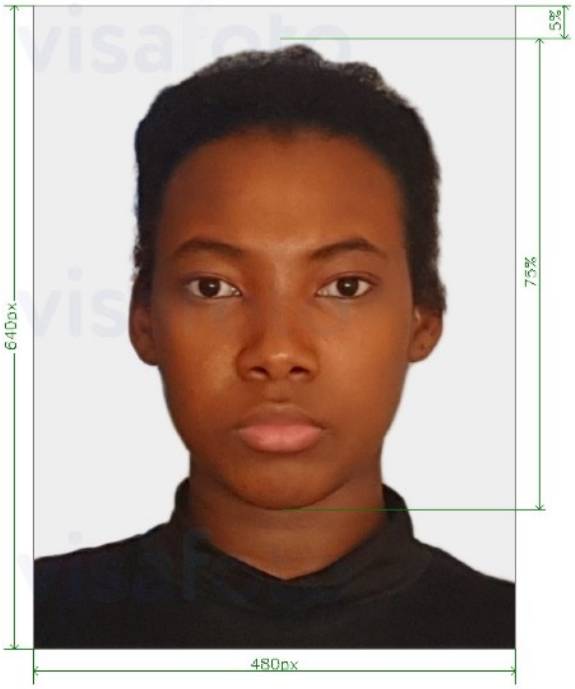
matches_joe.jpg.jpg



clrdcv2_matches_joe.jpg.jpg



chupik.jpg

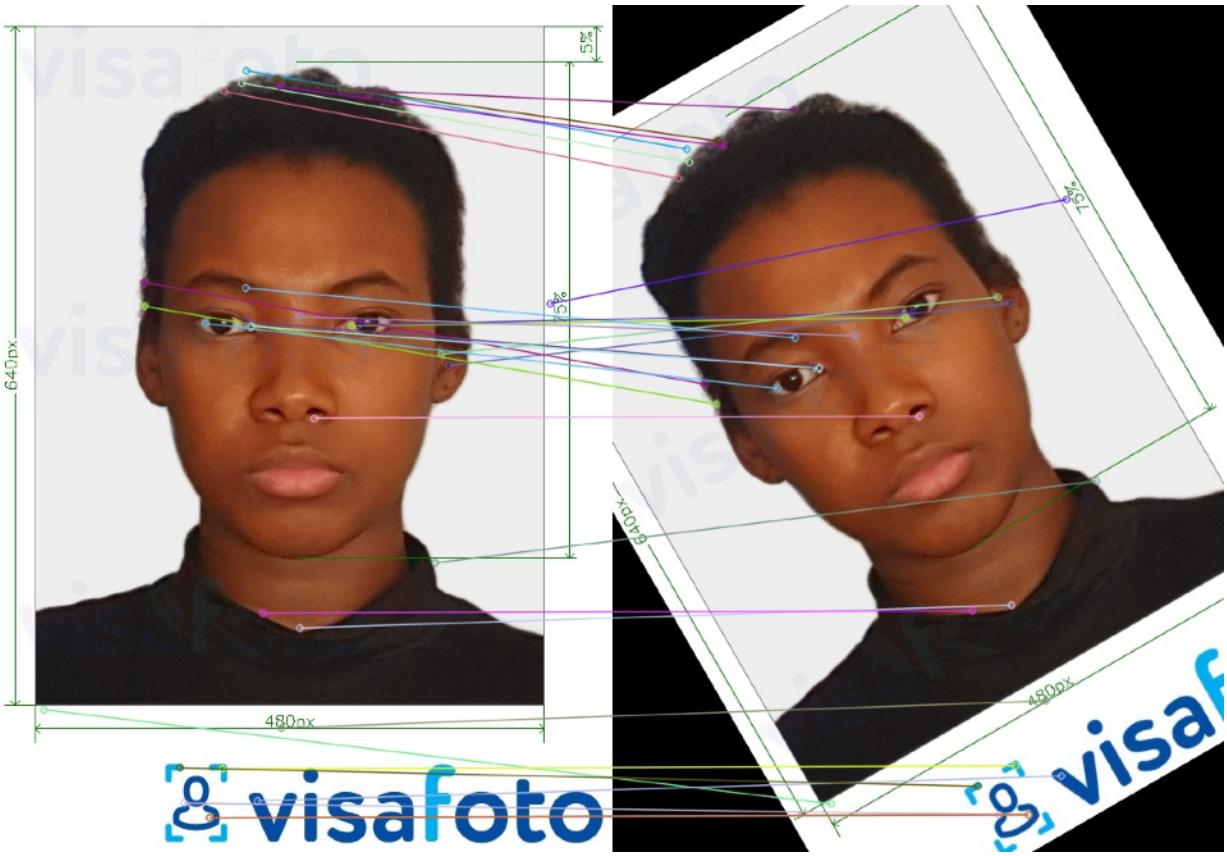


visafoto

matches_chupik.jpg.jpg



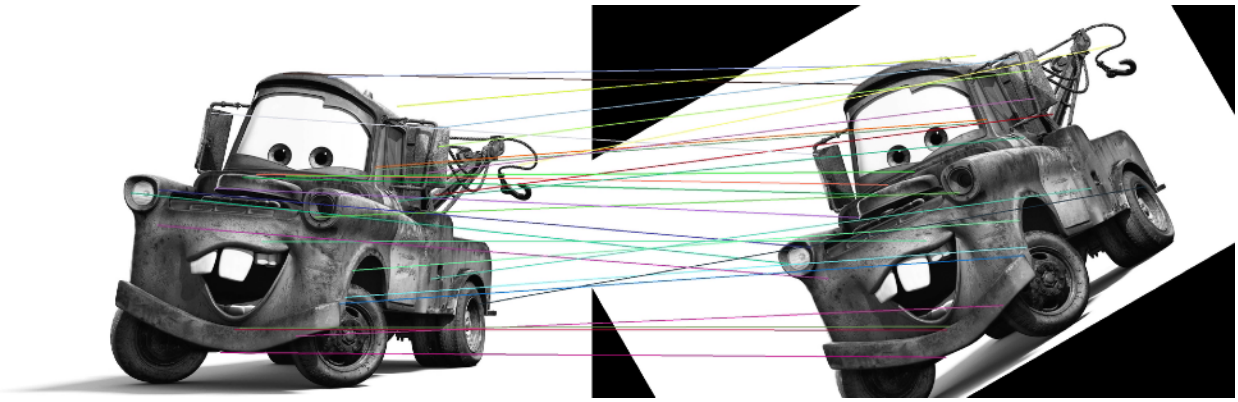
clrdcv2_matches_chupik.jpg.jpg



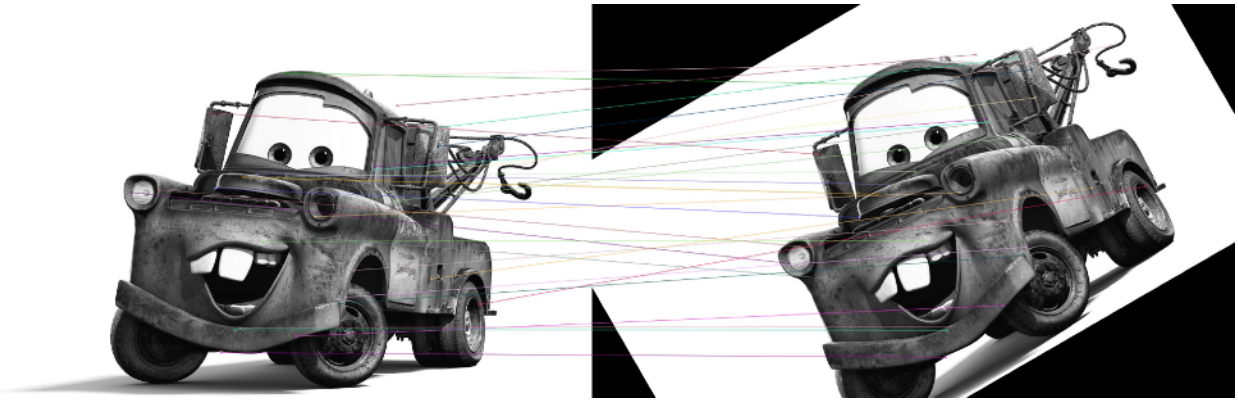
metr.jpg



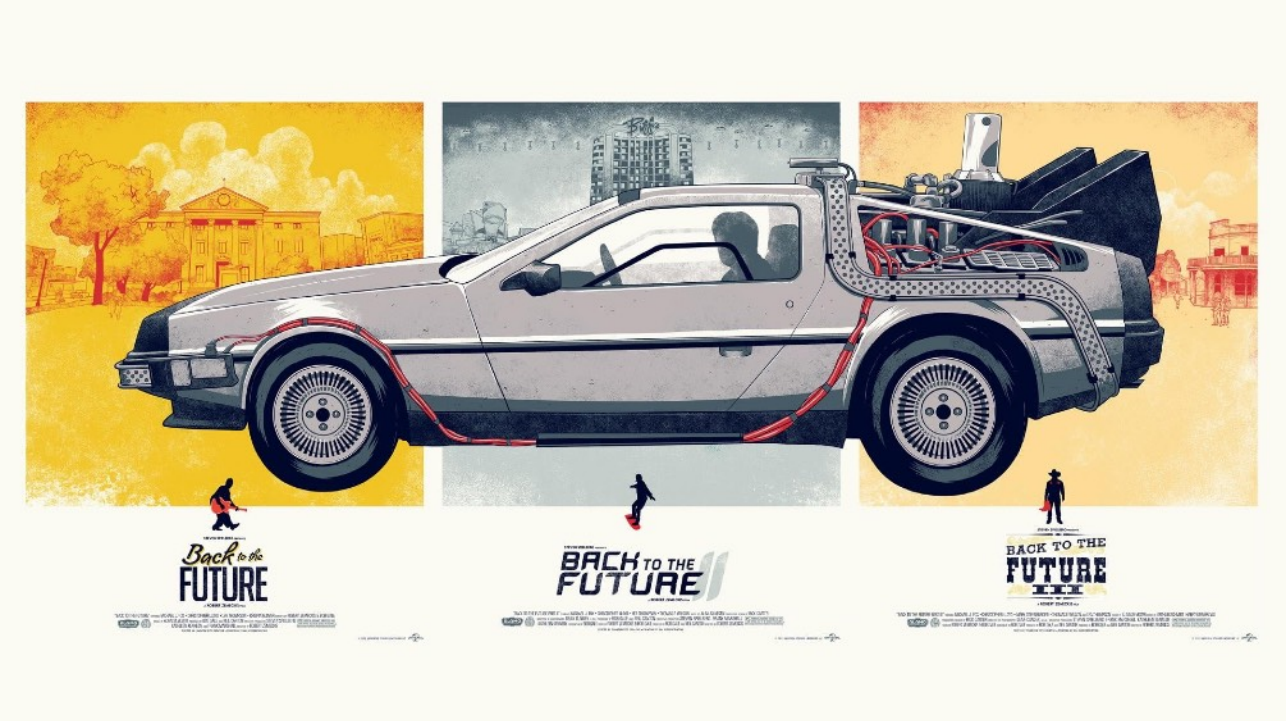
matches_metr.jpg.jpg



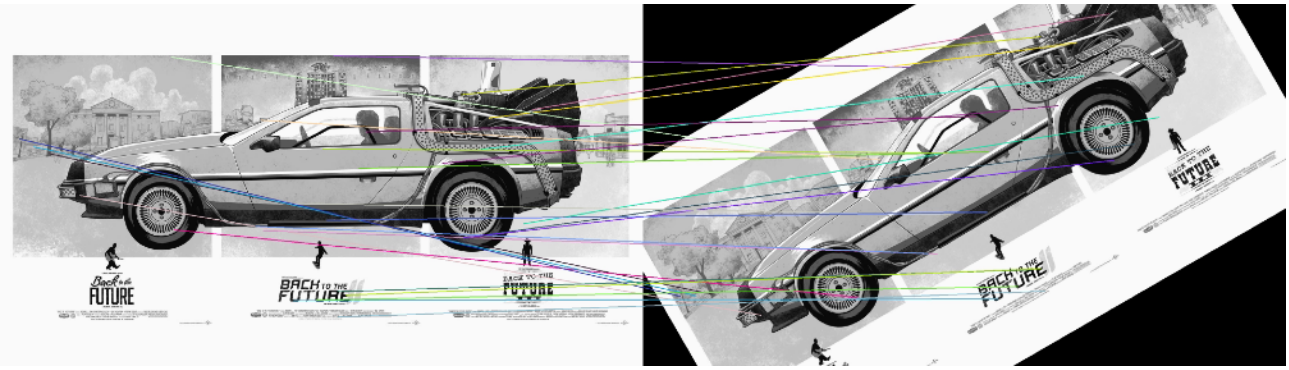
cv2_matches_metr.jpg.jpg



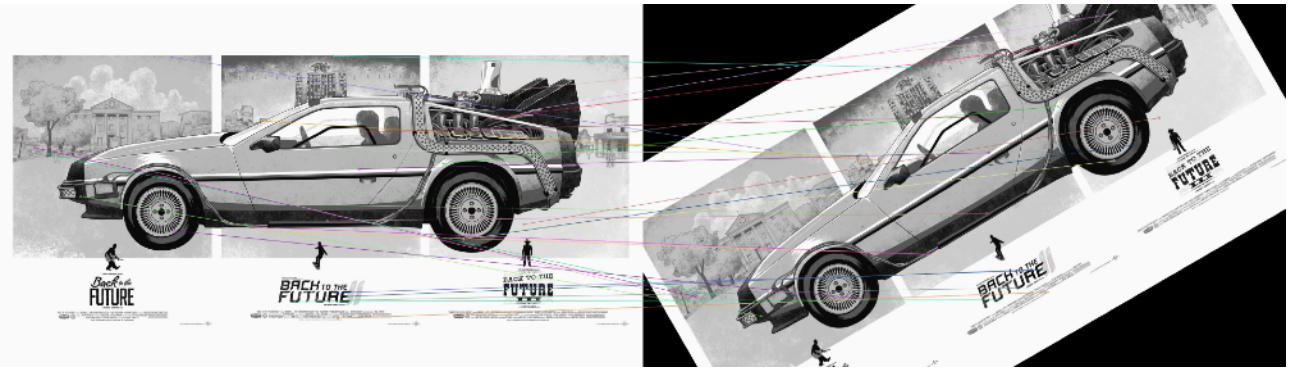
dlrn.jpg



matches_dlrn.jpg.jpg



cv2_matches_dlrn.jpg.jpg



Висновок:

Під час виконання даної лабораторної роботи, було використано дескриптор opencv і написано власну функцію метчингу. При порівнянні власного метчингу з тим, що пропонує opencv, зауважив, що хоч і точки можуть бути різними, точність метчингу є співмірною. Також було пеервірино метчинг на RGB версіях зображень, його точність не змінилась.

ДОДАТОК

lab2_finalcut.py

```
import cv2

import numpy as np

import random

import os


def compute_sift(image):

    if isinstance(image, str):

        image = cv2.imread(image, cv2.IMREAD_GRAYSCALE)


    sift = cv2.SIFT_create()

    keypoints, descriptors = sift.detectAndCompute(image, None)

    return keypoints, descriptors



def transform_image(image_path, angle):

    image = cv2.imread(image_path)

    # image = cv2.convertScaleAbs(image, alpha=1.5, beta=0)


    h, w = image.shape[:2]

    center = (w // 2, h // 2)

    matrix = cv2.getRotationMatrix2D(center, angle, 1.0)

    rotated_image = cv2.warpAffine(image, matrix, (w, h))


    return cv2.cvtColor(rotated_image, cv2.COLOR_BGR2GRAY)
```

```
def match_descriptors(des1, des2):
    matches = []

    for i, d1 in enumerate(des1):
        best_match = None
        best_distance = float('inf')
        for j, d2 in enumerate(des2):
            distance = np.linalg.norm(d1 - d2)
            if distance < best_distance:
                best_distance = distance
                best_match = (i, j, best_distance)
        if best_match:
            matches.append(best_match)

    matches.sort(key=lambda x: x[2])
    return matches



def draw_matches(img1, kp1, img2, kp2, matches, output_path):
    match_img = cv2.hconcat([cv2.cvtColor(img1,
cv2.COLOR_GRAY2BGR), cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)])
    h1, w1 = img1.shape

    for i, j, _ in matches[:30]:
```

```
        pt1 = tuple(map(int, kp1[i].pt))
        pt2 = tuple(map(int, kp2[j].pt))
        pt2 = (pt2[0] + w1, pt2[1])
        color = (random.randint(0, 255), random.randint(0, 255),
random.randint(0, 255))
        cv2.line(match_img, pt1, pt2, color, 2)

    cv2.imwrite(output_path, match_img)
    print(f"{output_path}")



img1_path = "original_photos/metr.jpg"
output_path = f"./matches_{os.path.basename(img1_path)}.jpg"

img1 = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)
img2 = transform_image(img1_path, 30)
print('images read')

kp1, des1 = compute_sift(img1_path)
print('sift1 ready')
kp2, des2 = compute_sift(img2)
print('sift2 ready')

matches = match_descriptors(des1, des2)
print('matches ready')
```

```
draw_matches(img1, kp1, img2, kp2, matches, output_path)
```

lab2_cv2.py

```
import cv2
import os



def compute_sift(image):
    if isinstance(image, str):
        image = cv2.imread(image)

    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(image, None)
    return keypoints, descriptors



def transform_image(image_path, angle):
    image = cv2.imread(image_path)

    h, w = image.shape[:2]
    center = (w // 2, h // 2)
    matrix = cv2.getRotationMatrix2D(center, angle, 1.0)
    rotated_image = cv2.warpAffine(image, matrix, (w, h))
    return cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB)
```

```
def match_descriptors(des1, des2):
    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)
    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda x: x.distance)
    return matches



def draw_matches(img1, kp1, img2, kp2, matches, output_path):
    match_img = cv2.drawMatches(img1, kp1, img2, kp2,
matches[:30], None,

flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    cv2.imwrite(output_path, cv2.cvtColor(match_img,
cv2.COLOR_RGB2BGR))
    print(f"Результат матчмейкінгу збережено в {output_path}")



img1_path = "original_photos/metr.jpg"
output_path = f"./
clrdcv2_matches_{os.path.basename(img1_path)}.jpg"

img1 = cv2.imread(img1_path)
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2 = transform_image(img1_path, 30)
print('images read')

kp1, des1 = compute_sift(img1)
```

```
print('sift1 ready')

kp2, des2 = compute_sift(img2)

print('sift2 ready')


matches = match_descriptors(des1, des2)

print('matches ready')

draw_matches(img1, kp1, img2, kp2, matches, output_path)
```